Solution: Problem Challenge 2

Problem Statement

You are given a 2D matrix containing only 1s (land) and 0s (water).

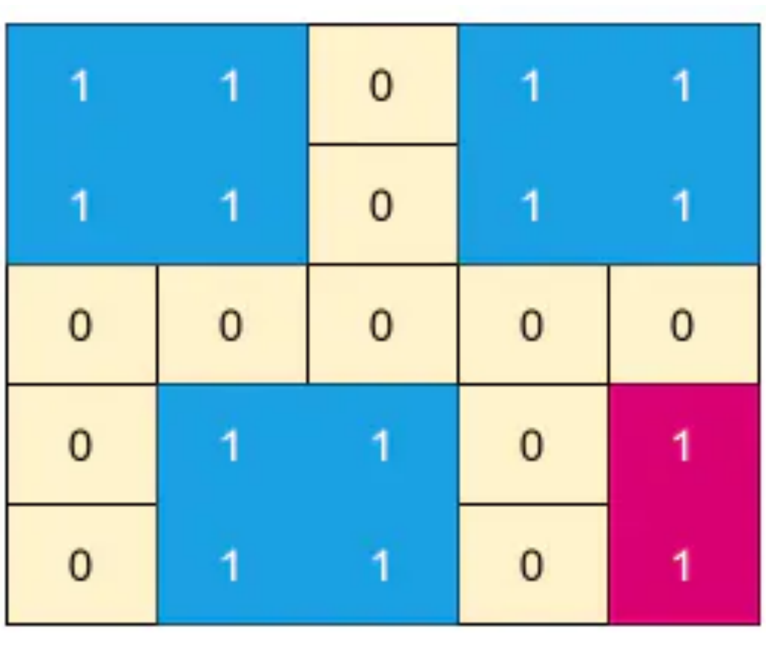
An **island** is a connected set of 1s (land) and is surrounded by either an edge or 0s (water). Each cell is considered connected to other cells horizontally or vertically (not diagonally).

Two islands are considered the same if and only if they can be translated (not rotated or reflected) to equal each other.

Write a function to find the number of distinct islands in the given matrix.

Example 1

Input: matrix =



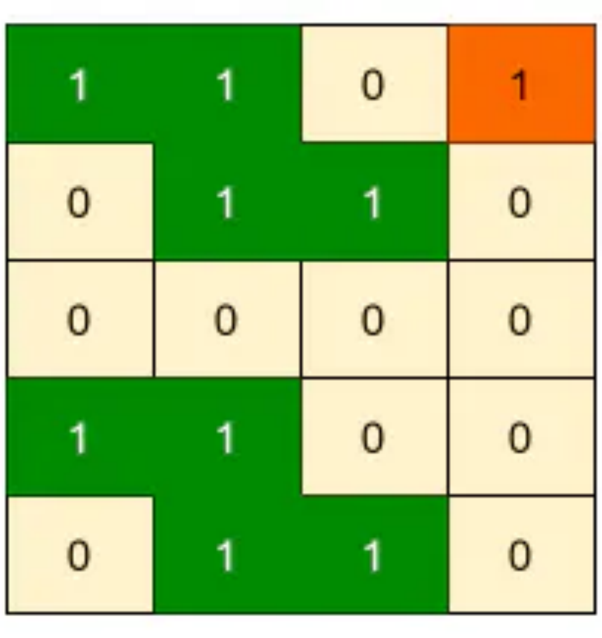
1	1	0	1	1
1	1	0	1	1
0	0	0	0	0
0	1	1	0	1
0	1	1	0	1

Output: 2

Explanation: There are four islands in the given matrix, but three of them are the same; hence, there are only two distinct islands.

Example 2

Input: matrix =



1	1	0	1
0	1	1	0
0	0	0	0
1	1	0	0
0	1	1	0

Output: 2

Explanation: There are three islands in the given matrix, but two of them are the same; hence, there are only two distinct islands.

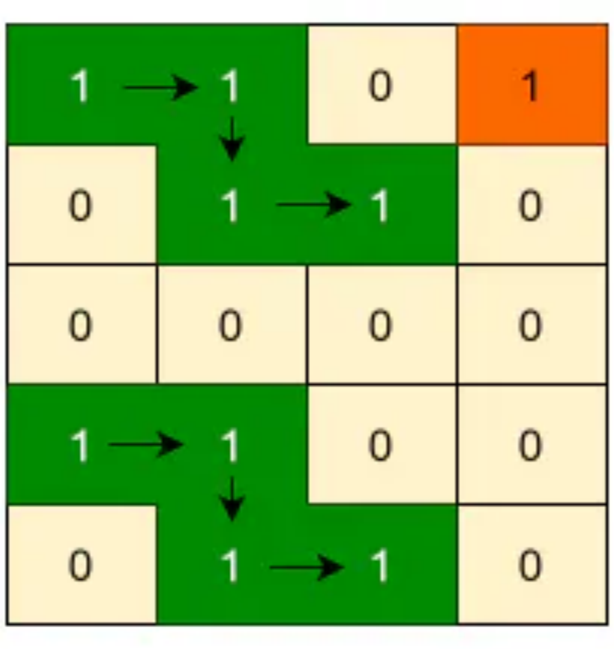
Solution

The question follows the Island pattern and is quite similar to Number of Islands problem.

We will traverse the matrix linearly to find islands. We can use the **Depth First Search** (**DFS**) or **Breadth First Search** (**BFS**) to traverse an island i.e., to find all of its connected land cells.

How do we decide if two islands are same?

If two islands are same, their traversal path should be same too. This means, if we perform a DFS or BFS on two equal islands starting from their top-left cell, the traversal pattern should be exactly same for both the islands. For example, here is the DFS traversal of the example-2 mentioned above:



We can clearly see that the two equal islands have same traversal pattern.

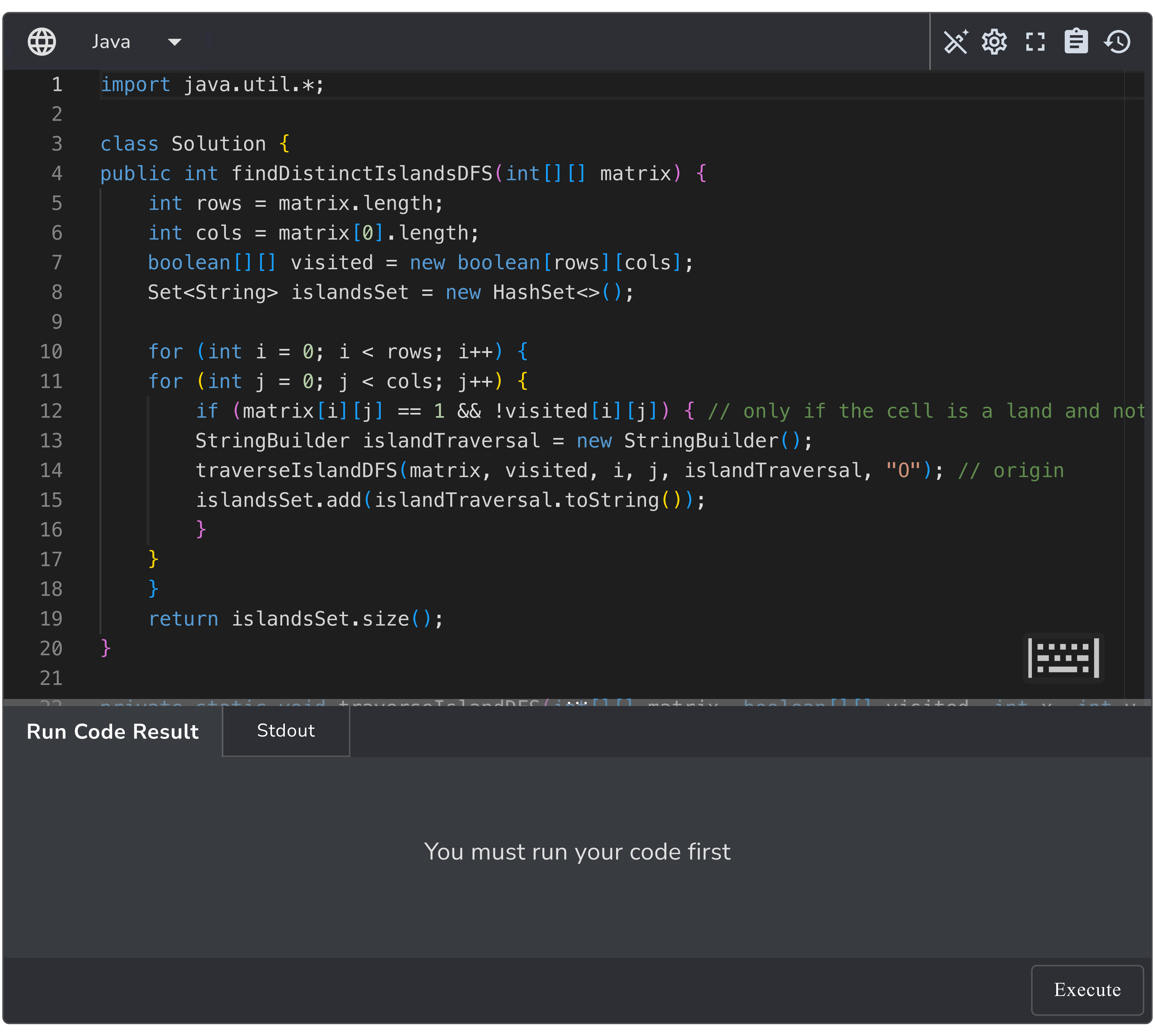
We can utilize this fact to develop an algorithm.

While traversing an island, we can construct a string that maps the traversal path of the island. For example, here is the DFS traversal of the two same islands mentioned in Example-2 ('R' for right move, 'D' for down move, 'O' for origin denoting the start): **ORDR**

We can start inserting these traversal strings of each island in a HashSet. This will ensure that we will not have any duplicate traversal string in the HashSet, thus giving us distinct islands. When we finish traversing the matrix, the HashSet will contain the distinct traversal path of all islands. Hence, the total number of elements in the HashSet will be equal to distinct number of islands.

Code

Here is the code for this algorithm:



```java
import java.util.*;

class Solution {
public int findDistinctIslandsDFS(int[][] matrix) {
    int rows = matrix.length;
    int cols = matrix[0].length;
    boolean[][] visited = new boolean[rows][cols];
    Set<String> islandsSet = new HashSet<>();

    for (int i = 0; i < rows; i++) {
    for (int j = 0; j < cols; j++) {
        if (matrix[i][j] == 1 && !visited[i][j]) { // only if the cell is a land and not
        StringBuilder islandTraversal = new StringBuilder();
        traverseIslandDFS(matrix, visited, i, j, islandTraversal, "O"); // origin
        islandsSet.add(islandTraversal.toString());
        }
    }
    }
    return islandsSet.size();
}

```

You must run your code first

Time Complexity

Time complexity of the above algorithm will be $O(M * N)$, where 'M' is the number of rows and 'N' is the number of columns of the input matrix. This is due to the fact that we have to traverse the whole matrix to find islands.

Space Complexity

DFS recursion stack can go M*N deep when the whole matrix is filled with '1's. Hence, the space complexity will be $O(M * N)$, where 'M' is the number of rows and 'N' is the number of columns of the input matrix.

← Previous
Problem Challenge 2 (Medium)

Next →
Problem Challenge 3 (Medium)

☐ Mark as Completed